Data Types

Data Types A core job of most programming languages is dealing with data. Everything you do in a computer program involves data in some way. Data in any programming language has different data types. Data types help programmers and their programs determine what they can and cannot do with a given piece of data. This chapter takes a quick tour of Python’s basic data types. Everything you do in Python involves data and data types.

Everything with a value in Python is said to be an object. Furthermore, each object has an associated data type or, more concisely, type, which has an associated class. For instance, the objects 42 and 123 are instances of the integer type, i.e., the int class. Similarly, the values True and False are instances of the Boolean type, i.e., the bool class.

In Python, the terms object and value can be used interchangeably, as can the terms class, data type, and type. We will use them that way, too.

Data Types Python has a large number of built-in data types compared to many other languages:

Data Type Class Category Kind Mutable integers int numerics Primitive No floats float numerics Primitive No boolean bool booleans Primitive No strings str text sequences Primitive No ranges range sequences Non-primitive No tuples tuple sequences Non-primitive No lists list sequences Non-primitive Yes dictionaries dict mappings Non-primitive Yes sets set sets Non-primitive Yes frozen sets frozenset sets Non-primitive No functions function functions Non-primitive Yes NoneType NoneType nulls –?– No This table shows just a fraction of the available data types. In this book, you will learn about all these types.

The first 4 data types are Python’s so-called primitive types. Primitive types are the most fundamental data types in a language. Almost all data in a language is built up from these primitive types. For instance, the sequence and mapping types may include primitives and complex types built from those primitives.

Some programmers think of None as a primitive type, but that’s debatable. We’re agnostic on the matter.

The sequences, mappings, and sets are all non-primitive types. In particular, they are also called collection types since each collects multiple items in a single object. We’ll refer to these types as collections.

Though functions are non-primitives, they are not collections.

The final column of the table shows which types are mutable and which are not. Mutable types are types whose objects can be altered after they are created. So, for example, we can add, remove, and replace elements in a list. Immutable types can’t be altered after creation. Instead, you must create a new object with a different value. We’ll explore this distinction and its subtleties in greater detail in the Variables as Pointers chapter.

In this chapter, we’ll focus on the primitive types. However, we’ll also make a nodding acquaintance with the non-primitive types. First, though, let’s talk about literals, and then we’ll meet the different primitive types.

Is every primitive in Python an object? Is every object in Python a primitive? Explain your reasoning.

Click here to ask LSBot this question Type your answer to the question above… (Press Cmd/Ctrl+Enter to send) Check My Answer Literals You can use literals to represent most data type values. A literal is any syntactic notation that lets you directly represent an object in source code. For instance, all of the following are literals in Python:

Copy Code ‘Hello, world!’ # str literal 3.141592 # float literal True # bool literal {‘a’: 1, ‘b’: 2} # dict literal [1, 2, 3] # list literal (4, 5, 6) # tuple literal {7, 8, 9} # set literal Not all objects have literal forms. In those cases, we use the type constructor to create objects of the type. For instance:

Copy Code range(10) # Range of numbers: 0-9 range(1, 11) # Range of numbers: 1-10 set() # Empty set frozenset([1, 2]) # Frozen set of values: 1 and 2 Numeric Values Numeric values represent numbers. Numbers can be added, subtracted, multiplied, and divided and can be used in a wide variety of mathematical operations. Python also supports other numeric types, such as complex, decimal, and fractional numbers.

Some programming languages treat integers and floating point numbers as a single numeric data type. Python uses different data types to represent numbers. Let’s talk about integers first.

Integers The integer data type (int) represents integers, i.e., the whole numbers (…, -3, -2, -1, 0, 1, 2, 3, …). Integers do not have a fractional (or decimal) part.

Examples of integer literalsCopy Code 1 2 -3 234891234 131_587_465_014_410_491 Note that you can’t use commas or periods for grouping: neither 123,456,789 nor 123.456.789 is valid. Instead, you can write the number without separators (123456789) or break up the number with underscores (123_456_789).

There is no pre-defined limit to the size of an integer. You can have integers with as many digits as will fit in memory, though you may not be able to do much with them then.

Consider the expression 3 + 2. When we execute this code, how many literals are involved? How many integers are involved? Explain why.

Click here to ask LSBot this question Type your answer to the question above… (Press Cmd/Ctrl+Enter to send) Check My Answer Floats The floating point data type (float) represents floating point numbers, aka, the real numbers. Floating point numbers include integers and numbers with digits after the decimal point.

Examples of float literalsCopy Code 1.0 1.4142 -3.14159 42348.912346 131_587_465.014_410_491 As with integers, you can’t use commas or periods for grouping: neither 42,348,912.346 nor 42.348.912,346 is valid. Instead, you can write the number with only a single decimal point (42348.912346) or break up the number with underscores (42_348.912_346).

Working With Big and Small Floats This section is optional. You probably won’t need this information unless you’re working on a math- or science-intensive application.

In theory, floats can represent any number, with or without a decimal point. In practice, there are limits that can be looked up in the sys.float_info module as follows:

Copy Code import sys

The maximum number of digits that can be accurately presented

print(sys.float_info.dig) # Typically 15

The largest possible positive float value

print(sys.float_info.max) # About 1.8 * (10**308)

The smallest non-zero positive float value

print(sys.float_info.min) # About 2.2 * (10**-308) Don’t worry about memorizing those values.

Note that 10**n, where n is positive, represents a 1 followed by n zeros. Thus, 10**308 is a 1 followed by 308 zeros; a truly enormous number. On the other hand, 10-**n is equivalent to the reciprocal of 10**n, e.g., 1.0 / 10**n. Thus, 10-**308 is a truly minuscule number.

Python prints extremely large and small floats using scientific notation:

Copy Code print(3.14 * (10**20)) # 3.14e+20 print(3.14 * (10**-20)) # 3.14e-20 You can read the e+20 as meaning 10 to the 20th power, and e-20 as the reciprocal of 10 to the 20th power.

You can also use scientific notation when writing float literals:

Copy Code print(3.14e+20 / 2.72e-15)) # 1.1544117647058823e+35 It’s worth pointing out that these floating point issues are not present when working with integers. Integers can be as small or large as you want, provided they fit in memory. Furthermore, integers don’t get printed with scientific notation.

Copy Code print(3 * (10**20)) # 300000000000000000000 If you want to use scientific notation to write a large integer, you must use the int function to convert the number to an integer:

Copy Code print(int(3e+20)) # 300000000000000000000 Can every reasonably small integer (like ones you’d typically use in programs) be represented as a float? How about vice versa? Why or why not?

Click here to ask LSBot this question Type your answer to the question above… (Press Cmd/Ctrl+Enter to send) Check My Answer Variables and Assignment Most programming languages use something called variables. Variables are merely names used to identify values (that is, data). In fact, they are also called identifiers, a more general term that applies to variables, constants, function names, class names, and more.

Variables in Python are usually created by initializing them to a value. For instance, consider the following code:

Copy Code pi = 3.141592653589793 life_the_universe_and_everything = 42 Here we’ve created two variables, pi and life_the_universe_and_everything, then initialized them to the values 3.141592653589793 and 42 respectively.

The syntax used when creating variables, e.g., foo = ‘abc’, is called assignment. We can say that the variable foo is assigned to the value ‘abc’ or that the value ‘abc’ is assigned to the variable foo. Both phrases mean the same thing, but the former is usually preferred. However, the latter phrasing is perfectly acceptable. We use both phrasings at Launch School, depending on what reads better in any particular context.

If a particular variable is being assigned a value for the very first time, we may call the assignment an initialization. If the variable has already been initialized, we should refer to the assignment as a reassignment. For example:

Copy Code foo = ‘Hello, world.’ # Assignment or initialization foo = “That’s all, folks!” # Reassignment We’ll discuss variables and assignment in much more detail later on.

What’s the minimum number of objects we must have in our program if we’re going to reassign a variable? Why?

Click here to ask LSBot this question Type your answer to the question above… (Press Cmd/Ctrl+Enter to send) Check My Answer Boolean Values Boolean values represent binary states such as true or false, on or off, yes or no, one or zero, and so on. The only Boolean values are True and False.

In Python, booleans sometimes act like numeric values: you can use them in mathematical and comparison expressions. You shouldn’t do that, really. However, you can always play with it in the REPL:

> 3 + True 4
>
> True == 1 True Later, we will discuss a related but different concept: truthiness. While Boolean values are also truthiness values, truthiness values often aren’t Boolean.

Suppose you want to represent the state of a light switch in your application. You can use Boolean values: True for on and False for off.

Copy Code toggle_on = True session_active = False Boolean values have a starring role when working with comparison operators. We’ll talk about comparisons in the next chapter. For now, it’s enough to know that comparisons check whether a value is equal to, less than, or greater than another value. They return a Boolean result.

> 5 == 5 True
>
> 100 < 99 False Text Sequences Text sequences are strings of characters, such as words, sentences, dates, foreign text, and so on. They also include things like byte strings, which are sequences of byte-sized values. We don’t mention byte strings again in this book.

Example text sequence valuesCopy Code ‘Hello!’ “He’s pining for the fjords!” ‘1969-07-20’ A string is a text sequence of Unicode characters. Developers often work with text data like names, messages, and descriptions. Python uses strings to represent such data.

Text sequences can often be treated as ordinary sequences. For instance:

Copy Code greeting = ‘hello’ for letter in greeting: print(letter) # h # e # l # l # o There is one significant difference between a text sequence and an ordinary sequence. Ordinary sequences contain zero or more objects, but a text sequence does not contain any objects: it simply contains the characters (or bytes) that make up the text sequence. Those characters or bytes are not objects; they are simply part of the value.

Proving that the characters of a string are not stored as objects requires studying the Python source code. We’re not going to do that.

Based on Python’s treatment of strings, what arguments could you make both for and against calling a string a collection?

Click here to ask LSBot this question Type your answer to the question above… (Press Cmd/Ctrl+Enter to send) Check My Answer String Literals and Dealing With Quotes You can write string literals with single, double, or triple quotes.

Copy Code ‘Hi there’ # Single quotes “Monty Python’s Flying Circus” # Double quotes

Triple single quotes

’‘King Arthur: “What is your name?” Black Knight: “None shall pass!” King Arthur: “What is your quest?” Black Knight: “I have no quarrel with you, but I must cross this bridge.”’’

Triple double quotes

“““Man:”Is this the right room for an argument?” Other Man: “I’ve told you once.” Man: “No you haven’t!” “”” All of these forms create strings in the same way. Most Python programmers use single quotes as the primary way to write string literals but turn to double quotes when the string’s value contains single quotes. However, triple quotes are often used for multi-line strings and a special kind of comment that we won’t get into.

You may sometimes need a string that contains both single and double quotes. For instance, if you want to print My nickname is “Wolfy”. What’s yours?, you must consider the fact that the string has both single and double quotes. You can use triple quotes with such a string, or you can escape one of the quote characters like so:

Copy Code print(“““My nickname is”Wolfy”. What’s yours?“““) print(‘My nickname is “Wolfy”. What’s yours?’) print(”My nickname is "Wolfy". What’s yours?“) The backslash, or escape character (), tells the computer that the next character isn’t syntactic but is part of the string. Escaping a quote prevents Python from seeing it as the string’s end. Pay attention to the type of slash you use: a forward slash (/) has no significance between quotes. If you want a literal backslash in your string, you must double up the backslashes or use a so-called raw-string literal (we’ll discuss raw strings shortly):

Copy Code print(“C:\Users\Xyzzy”) # Each \ produces a literal
Indexing Strings You can access the individual characters in a string with the [] indexing syntax. The value between the brackets must be an integer between 0 and the length of the string minus 1:

> my_str = ‘abc’ my_str[0] a
>
> my_str[1] b
>
> my_str[2] c
>
> my_str[4] IndexError: string index out of range You can also use negative integers to access characters based on the distance from the end of the string. For instance, my_str[-1] returns the last character in the string, while my_str[-2] returns the next to last character. The index of the first character is given by -len(my_str):
>
> my_str = ‘abc’ my_str[-1] c
>
> my_str[-2] b
>
> my_str[-3] a
>
> my_str[-4] IndexError: string index out of range If we know the length of a string, how could we access the first character of the string without using my_string[0]?

Click here to ask LSBot this question Type your answer to the question above… (Press Cmd/Ctrl+Enter to send) Check My Answer Raw Strings and f-Strings String literals with an r prefix are raw string literals. Raw string literals don’t recognize escapes, so you can use literal characters freely.

Copy Code # Both of these print C: print(“C:\Users\Xyzzy”) # Each \ produces a literal
print(r”C:“) # raw string literal Raw strings are most often used with Windows-style file names and regular expressions. We cover regular expressions in our Regular Expressions book.

Another prefix you can use with string literals is f. It’s a recent Python addition for defining formatted string literals or, more succinctly, f-strings. F-strings enable an operation called string interpolation. You create an f-string by preceding the opening quote with the letter f:

> f’5 plus 5 equals {5 + 5}.’ ‘5 plus 5 equals 10.’
>
> my_name = ‘Karl’ f’My name is {my_name}.’ ‘My name is Karl.’
>
> my_name = ‘Clare’ greeting = ‘Ey up?’ f’{greeting} My name is {my_name}.’ Ey up? My name is Clare. String interpolation is a handy way to merge Python expressions with strings. The basic syntax is:

Copy Code f’Blah {expression} blah.’ Python replaces the {expression} substring with the value of the expression inside the braces: it interpolates the expression’s value. You’ll learn to love this feature; it saves time and effort. However, remember that string interpolation is a feature of f-strings only. It doesn’t work if you forget the f or omit the braces.

You may include as many {expression} substrings as needed in an f-string. If you need literal { or } characters in an f-string, you can escape them by doubling the { and } characters you want to use as literal characters:

Copy Code foo = ‘hello’ print(f’Use {{foo}} to embed a string: {foo}’) # Use {foo} to embed a string: hello In the above example, we have an f-string that embeds both {{foo}} and {foo}. The former gets interpreted as the literal string {foo}, while the latter gets replaced by the value of foo.

We have two more f-string variants. Earlier in this chapter, we discussed the use of underscores in numbers to make them more readable, e.g., 12_345_678. However, if you print a number, no matter its size, it won’t be printed with underscores (or even commas) unless you use an appropriate f-string:

Copy Code print(f’{123456789:_}’) # 123_456_789 print(f’{123456789:,}’) # 123,456,789 If you use this special form of f-string on a floating point number, only the part before the decimal point will be formatted as expected:

Copy Code print(f’{123456.7890123:_}’) # 123_456.7890123 print(f’{123456.7890123:,}’) # 123,456.7890123 Functions Functions are chunks of standalone, reusable Python code designed to perform an action. They provide a way to break up your program into modular bits that enable a high degree of code reuse.

One thing that may surprise us is that Python functions have a data type. Many, but not all, languages share this characteristic. We won’t discuss this characteristic in this book. However, we will do so in the Core Curriculum.

None: When You Want Nothing In programming, we need a way to express the absence of a value. In Python, we do this with None. It can also indicate missing, unset, or unavailable data and may sometimes be an error indication.

None is a literal whose value is the lone representative of the NoneType class.

How would you describe the difference in meaning between False and None?

Click here to ask LSBot this question Type your answer to the question above… (Press Cmd/Ctrl+Enter to send) Check My Answer Sequences Sequences represent an ordered collection of objects. A sequence’s objects can be accessed using a numeric index. In many other languages, the array is the best-known sequence type, but Python uses lists to fill the same role. However, tuples and ranges are also important types.

Lists and Tuples Lists and tuples are so similar that we’ll cover them together.

Python organizes information into ordered lists using lists and tuples. Lists and tuples may contain any objects.

How do I pronounce tuple?

Most people seem to believe that tuple rhymes with couple and supple: TUHP-ul. However, many others say it rhymes with “Mott the Hoople”: TOOP-ul. Finally, mathematically-inclined people say it should rhyme with quadruple: TOO-pel. The author tends to switch back and forth between the last two, but Launch School is okay with all 3 pronunciations.

List literals use square brackets [] surrounding a comma-delimited list of values, while tuples use parentheses (). The comma-delimited list values are known as elements. Let’s make a list and a tuple:

> my_list = [1, ‘xyz’, True, [2, 3, 4]] my_list [1, ‘xyz’, True, [2, 3, 4]]
>
> tup = (‘xyz’, [2, 3, 4], 1, True) tup (‘xyz’, [2, 3, 4], 1, True) Notice that both objects contain a mix of element types, including another list ([2, 3, 4]).

List and tuple literals may also use a multi-line format, which is especially useful for more extensive or nested collections:

Copy Code [# Begin multi-line list literal “Monty Python’s Flying Circus”, (# Begin multi-line tuple literal ‘Eric Idle’, ‘John Cleese’, ‘Terry Gilliam’, ‘Graham Chapman’, ‘Michael Palin’, ‘Terry Jones’,), # End multi-line tuple literal] # End multi-line list literal The trailing comma on the last items in the list and tuple are optional and common practice when writing multi-line collection literals, such as lists, tuples, and dictionaries. Try to be consistent if you use it. It should only be used in multi-line literals.

You can access objects in a list or tuple with the [] indexing syntax. The value between the brackets must be an integer between 0 and the length of the sequence minus 1:

> my_list = [1, ‘xyz’, True, [2, 3, 4]] my_list[0] 1
>
> my_list[1] ‘xyz’
>
> my_list[2] True
>
> my_list[3] [2, 3, 4]
>
> my_list[4] IndexError: list index out of range A crucial distinction between lists and tuples is that lists are mutable. Tuples, however, are immutable.

One way to mutate a list is to use the indexing syntax to the left of the = in an assignment. This is called indexed reassignment or element reassignment. (We usually prefer element reassignment, but you will often hear and read about indexed reassignment.) For instance:

> my_list[3] = ‘New value’ my_list [1, ‘xyz’, True, ‘New value’]
>
> tup[3] = ‘New value’ TypeError: ‘tuple’ object does not support item assignment The immutability of tuples limits their usefulness slightly, but Python can perform more optimizations on tuples. The optimizations can reduce storage requirements and improve performance.

Here’s a bit of Python weirdness: if you want to use a literal to create a tuple with one element, you can’t simply write:

Copy Code my_tuple = (1) print(type(my_tuple)) # <class ‘int’> In this case, the parentheses cause Python to treat the expression as a parenthesized expression, not as a tuple literal. To define a one-element tuple, you must place a comma after the element value:

Copy Code my_tuple = (1,) print(type(my_tuple)) # <class ‘tuple’> The most important facts to remember about lists and tuples are:

The order of the elements is significant. Lists are mutable; tuples are immutable. Use indexing syntax to retrieve specific elements. Use indexing syntax to reassign specific list elements. Index numbers are non-negative integers starting from 0 and ending at the sequence’s length minus 1. (Later, we will learn that negative integers are also allowed. For now, though, we’ll only use non-negative integers.) Do you expect negative indexing to work with lists and tuples in the same way it works with strings? Why or why not?

Click here to ask LSBot this question Type your answer to the question above… (Press Cmd/Ctrl+Enter to send) Check My Answer Ranges A range is a sequence of integers between two endpoints. Ranges are most commonly used to iterate over an increasing or decreasing range of integers. Let’s see how we can create some ranges:

> tuple(range(5)) (0, 1, 2, 3, 4)
>
> tuple(range(5, 10)) (5, 6, 7, 8, 9)
>
> list(range(1, 10, 2)) [1, 3, 5, 7, 9]
>
> list(range(0, -5, -1)) [0, -1, -2, -3, -4] With one argument, the range starts from 0 and ends just before the argument. Thus, range(5) goes from 0 through 4.

With two arguments, the first argument is the starting point, while the second argument is one greater than the last number in the range. Thus, range(5, 10) goes from 5 through 9.

With three arguments, the 3rd argument is a step value. Thus, range(1, 10, 2) goes from 1 through 9 with a step of 2 between the numbers. On the other hand, range(0, -5, -1) goes from 0 through -4 with a step of -1. This results in a range that goes from the highest value to the lowest.

Python optimizes ranges to save memory; it doesn’t need the integers before a program asks for them. One way to get those numbers is to convert the range to a list or tuple. This is why we used the list and tuple functions to expand each range; either will suffice.

Like strings, lists, and tuples, you can use indexing syntax to access specific numbers in a range object:

> my_range = range(5, 10) my_range[3] # 8 Mappings Mappings represent an unordered collection of objects stored as key-value pairs. Each key (usually a string) provides a unique identifier for a specific object in the mapping. The value is the object associated with that key.

The dictionary (class dict) is the most common mapping in Python. It’s the only kind of mapping we’ll see in this book.

A dictionary (dict) is a collection of key-value pairs. A dict is similar to a list but uses keys instead of indexes to access specific elements. Other languages use different names for similar structures: hashes, mapping, objects, and associative arrays are the most common terms. Essentially, a Python dictionary is a collection of key-value pairs.

You can create dictionaries using dict literals. They have zero or more key-value pairs separated by commas, all embedded within curly braces ({}). A key-value pair associates a key with a specific value. Key-value pairs in dict literals use the key followed by a colon (:) and then the value.

That sounds more complicated than it is. Let’s put those words into action. Type along!

Our example creates a dict with three key-value pairs:

> my_dict = { … ‘dog’: ‘barks’, … ‘cat’: ‘meows’, … ‘pig’: ‘oinks’, … } {‘dog’: ‘barks’, ‘cat’: ‘meows’, ‘pig’: ‘oinks’} This dict contains 3 keys: the strings ‘dog’, ‘cat’, and ‘pig’. The ‘dog’ key is mapped to the string ‘barks’ while ‘cat’ is mapped to ‘meows’

Dictionary literals may also use a multi-line format, which is especially useful for larger or nested dictionaries:

Copy Code my_dict = { ‘title’: “Monty Python’s Flying Circus”, ‘cast’: [‘Eric Idle’, ‘John Cleese’, ‘Terry Gilliam’, ‘Graham Chapman’, ‘Michael Palin’, ‘Terry Jones’,], ‘first_season’: 1969, ‘last_season’: 1974, ‘reboot_season’: None, } You can access objects in a dict with the [] key access syntax. The value between the brackets must be one of the keys in the dict:

> my_dict = { … ‘dog’: ‘barks’, … ‘cat’: ‘meows’, … ‘pig’: ‘oinks’ … } … my_dict[‘cat’] ‘meows’
>
> my_dict[‘bird’] KeyError: ‘bird’ You can use the same key access syntax on the left side of an assignment to replace the associated key’s value with a different object:
>
> my_dict[‘cat’] = ‘purrs’ my_dict {‘dog’: ‘barks’, ‘cat’: ‘purrs’, ‘pig’: ‘oinks’} As of Python 3.7, dictionary key/value pairs are stored in the same order they are inserted into a dictionary; before 3.7, the key/value pair order was unpredictable. This can be interpreted as meaning that dictionaries are ordered. However, even with this ordering, the key/value pair order can change. For instance:

Copy Code dic = {} dic[‘a’] = 1 dic[‘b’] = 2 print(dic) # {‘a’: 1, ‘b’: 2} del dic[‘a’] dic[‘a’] = 1 print(dic) # {‘b’: 2, ‘a’: 1} For this reason, we prefer to categorize dictionaries (and other mappings) as unordered collections. Dictionaries, in particular, are unordered collections in which insertion order is preserved.

One last note: You can use almost any immutable object as a key in a dict; it doesn’t have to be a string. The only significant requirement for keys is that they are hashable. We discuss hashing in our PY110 course. For now, though, the important concept is that immutable types are almost always hashable, while mutable types are almost always non-hashable. Thus, since lists are mutable and, therefore, unhashable, you can’t use a list as a dictionary key. However, you can use a tuple as a dictionary key provided all of its elements are immutable and hashable.

Give an example of a situation where a dictionary would be a better choice than a list. Conversely, describe a case where a list would be more appropriate than a dictionary.

Click here to ask LSBot this question Type your answer to the question above… (Press Cmd/Ctrl+Enter to send) Check My Answer Sets Sets represent an unordered collection of unique objects; the objects are sometimes called the members of the set. Sets are similar to mappings, except instead of using keys and values, a set is simply a collection of immutable (and hashable, as mentioned above) objects.

As the name suggests, sets support several operations corresponding to mathematical sets: unions, intersections, etc. We won’t discuss those in this book.

The literal syntax for sets is a comma-separated list of object values between curly braces ({}). As a special case, empty sets must be created with the set constructor since {} by itself is an empty dictionary. Let’s see how it’s done:

> d1 = {} # Empty dict print(type(d1)) <class ‘dict’>
>
> s1 = set() # Empty set print(s1) set()

Create a set from a literal

> s2 = {2, 3, 5, 7, 11, 13} print(s2) {2, 3, 5, 7, 11, 13}

Create a set from a string

> s3 = set(“hello there!”) {‘t’, ‘o’, ‘e’, ‘l’, ’ ‘, ’h’, ‘!’, ‘r’} In the last example, notice that the set only contains one occurrence of each character, even though the string has repeated instances of some characters. Set members are always unique, even if you try to add duplicates. This example also shows that the order of the characters in the set has nothing to do with the order of the characters in the string.

Set literals may also use a multi-line format, which is especially useful for larger sets:

Copy Code monty_python_cast = { ‘Eric Idle’, ‘John Cleese’, ‘Terry Gilliam’, ‘Graham Chapman’, ‘Michael Palin’, ‘Terry Jones’, } There are two main set types: ordinary sets (class set) and frozen sets (class frozenset). Frozen sets are merely immutable sets. They also lack a literal syntax: you must use the frozenset function to create one.

> s5 = frozenset([1, 2, 3]) print(s5) frozenset({1, 2, 3})
>
> s6 = frozenset((1, 2, 3)) print(s6) frozenset({1, 2, 3})
>
> s7 = frozenset({1, 2, 3}) print(s7) frozenset({1, 2, 3})
>
> s8 = frozenset(range(1, 4)) print(s8) frozenset({1, 2, 3}) Note that you can initialize a set or frozen set with any iterable type. In the most recent examples, we initialize a frozen set using a list, a tuple, a set, and a range. However, when printing frozen sets, they are always represented as using a set to initialize the frozen set.

One last note: You can use almost any immutable value as a set member. The only significant requirement is that the objects must be hashable.

How are sets different from lists? How are they the same?

Click here to ask LSBot this question Type your answer to the question above… (Press Cmd/Ctrl+Enter to send) Check My Answer Summary We’ve learned about Python’s fundamental data types in this chapter. We’ve learned how to create these objects but not how to use them. That’s our task in the next chapter. First, though, we have a few exercises for you.